Functions in JavaScript

A **function** is a block of code that performs a specific task. Instead of repeating the same code again and again, you can write it once in a function and call it whenever needed.

---

1. Function Declaration

This is the most common way to define a function.

```
function greet() {
  console.log("Hello, JavaScript!");
}

greet(); // Call the function
```

---

2. Function with Parameters

You can pass **data into functions** using parameters.

```
function greetUser(name) {
  console.log("Hello, " + name);
}

greetUser("Ali"); // Output: Hello, Ali
```

---

3. Function with Return Value

Functions can **return** values using the `return` keyword.

```
function add(a, b) {
  return a + b;
}

let result = add(5, 3); // 8
console.log(result);
```

---

4. Function Expressions

Functions can also be stored in variables.

```
const sayHi = function() {
  console.log("Hi there!");
};

sayHi();
```

---

5. Arrow Functions (ES6+)

A shorter way to write functions.

```
const square = (num) => {
  return num * num;
};

console.log(square(4)); // 16
```

One-liner version (if only returning a value):

```
const multiply = (a, b) => a * b;
console.log(multiply(2, 3)); // 6
```

---

6. Scope (Simple Explanation)

Variables declared inside a function are **local** and can't be accessed outside.

```
function showAge() {
  let age = 25;
  console.log(age);
}

showAge();
// console.log(age); // Error: age is not defined
```

---

7. Why Use Functions?

- Organize code into reusable pieces
- Avoid repetition
- Make your code cleaner and easier to understand
- Useful in handling events and user interactions

Summary

- Use `function` to define reusable code blocks.
- Pass data using parameters and return results with `return`.
- Store functions in variables or use arrow functions for brevity.
- Functions help you write cleaner and modular code.